김00 Java Backend Developer

Contact

2000.00.00 | 주소 | example@example.com

깃허브주소 | 기술 블로그 주소

School

0 0 대학교 졸업 (2000.00 ~ 2000.00)

정보통신공학과

0 학점 / 0학점

집요한 실천주의 개발자 000입니다.

- 왜? 라는 질문으로 근본을 파고들어 기술적, 소통적으로 문제를 해결한 경험이 있습니다.
- 단순 구현 보다 방법론과 그 근거를 탐구하는 대화를 선호합니다.
- 새로운 개념을 학습할 때 이론에 매몰되기 보다는 공식문서를 기반으로 실제 데모 프로젝트를 만들어 학습합니다.
- 명확한 의견을 가지고 있지만, 피드백은 빠르게 받아들이고 개선하는 데 주저하지 않습니다.

Skills

Java / Spring Boot

- **(주)소프트스퀘어드에서 외주를 진행할 당시** 동적쿼리와 페이지네이션를 활용한 **필터링 조회 API를 설계 및 개발**
- 객체지향 프로그래밍을 이해하고 **OOP** 와 **SOLID** 원칙에 따라 인터페이스 등을 활용한 개발 및 **코드 리팩토링**
- JVM 내부 동작을 이해하고, 내부 Heap 구성과 Garbage Collection 에 대한 이해 보유
- 불필요한 로그 대신 **IDE 디버깅 도구**를 활용해 버그를 찾고 코드 개선
- **RESTful API 개발** 및 @RestControllerAdvice **와 Enum** 을 통한 에러 정의로 예외처리를 중앙화한 경험
- 함수형 인터페이스를 통한 동적 파라미터를 사용하여 Lambda 및 Stream 을 통해 가독성과 재사용성을 향상
- Spring Security 의 SecurityFilterChain 를 활용하여 JWT 기반 인증 처리 구현 경험

MySQL / PostgreSQL

- 다수의 쿼리를 JOIN 을 통해 하나의 쿼리로 최적화하여 필터링 쿼리 **조회 성능을 20.82% 개선**한 경험
- **ACID 와 같은 RDBMS 의 특징을 이해**하고 다수의 JOIN, LIKE 등을 활용한 **복잡한 쿼리 작성 가능**
- **스키마를 갖춘 RDBMS 와 비정형 데이터를 처리 가능한 NoSQL 의 차이점**을 이해

Docker & AWS & Github Actions

- VM 과 Container 의 차이점과 사용 이유를 이해하고 Dockerfile 작성하여 Docker 사용 가능
- **CI/CD 의 개념과 필요성, Github Actions 의 장점**에 대한 이해 보유
- 네트워크 지식을 기반으로 AWS VPC 및 서브넷 구성을 구성하고 EC2, S3 등 자원 구축 가능

Projects

소프트스퀘어드 — Gridge Worker

> 사용자와 개발자를 매핑해주는 외주 공고 플랫폼인 소프트스퀘어드 사내에서 사용하는 어드민 페이지 개발

2024.04 ~ 2024.06 **(2개월)** | 5명 | 역할 : **Backend** | 🔗 Private Repository

- 기존 개발되어있던 몇백개의 클래스 가진 Spring 프로젝트 코드를 분석 후 컨벤션과 기존 로직을 활용한 추가 기능 개발
- 피그마 기획안과 ERD 를 분석하여 도메인을 이해하고, 패키지 구조 설계와 데이터 검증 과정을 거쳐 API를 직접 설계함
- 특정 데이터가 필요할때마다 **Swagger** 문서를 통해 필요한 API 검색 및 스펙에 맞추어 개발
- 테스트 데이터 적재를 위해 직접 DB 에 접근하지 않고, 다수의 API 를 순차적으로 호출하여 데이터 정합성을 고려한 적재

핵심 기능 1. 다중 조건 필터링 조회 API 개발

> 특정 프로젝트에서 **작업자별 업무 이력**을 필터링하여 **AI 가 초과근무 여부, 잘못된 입력 검출, 급여 계산**을 수행하는 기능

전체 데이터 조회

디자이너만 조회

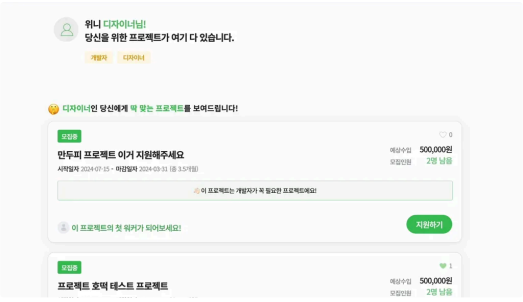
기술적 세부사항

- **QueryDSL** 를 통해 다수의 쿼리를 JOIN 하여 **하나의 동적 쿼리로 최적화**하여 필터링 쿼리 **조회 성능을 6.55% 개선**
- 3-Layered 아키텍쳐에 맞춰 Controller 계층에 예외 및 검증처리와 비지니스를 위한 Facade 계층 추가로 순환참조 방지
- 다수의 다양한 직군의 작업자별 업무 진행상황 처리를 위해 **Parallel Stream API** 활용하여 데이터 변환 혹은 처리
- 작업시간에 따라 작업자에게 급여가 지급되기에 소숫점 작업시간 계산을 위한 **SQL TIMESTAMPDIFF 와 ROUND** 사용
- 컨트롤러 레벨에서 일관된 기본값을 설정하여 코드 중복을 최소화하기 위해 **@PageableDefault** 로 페이지네이션 구현

핵심 기능 2. Next.js 으로 구현된 기능을 Spring 으로 전환하여 사용 환경 개선

> 하트 버튼 클릭 시 **관심 프로젝트로 등록**하고, 마이페이지에서 관심 공고 목록을 조회할 수 있는 기능

관심 프로젝트로 등록되어 있지 않은 상태



하트 버튼 클릭 시 관심 프로젝트로 등록

기술적 세부사항

- 비일관된 패키지 구조와 코드 스타일로 재사용성이 없는 Next.js 로직 일부를 간략한 **Spring Boot 코드로 마이그레이션**
- 멱등성을 보장하기 위해 **PUT 메서드**를 사용해 좋아요 상태를 관리함

MUSIC-IN-MY-DIARY

> **사용자 프롬프팅을 기반으로 음악을 추천하거나 가사를 생성하는 AI 기반 웹 애플리케이션**

2024.10 ~ 2024.11 **(1개월)** | 2명 | 역할 : **Backend & Frontend, 팀장** | 🔗 Frontend | 🔗 Backend

- 음원 사이트 정보 크롤링 및 사용자 요청에 따른 응답을 생성하는 Fast API 서버와 인증 관련 처리를 위한 Spring 서버 분리
- Spring MVC 아키텍쳐 상에서 Servlet, HandlerMapping 등과 Spring Security 동작 원리와 RestClient 설정 학습
- 단순 프로젝트 개발이 아닌 확장성과 유지보수성을 고려한 **클린 코드**를 높은 목표로 설정하여 **다양한 재사용성 패턴들을 적용**

핵심 기능 1. 인증 프로세스 구현

> 로그인한 사용자만 채팅 기능, 마이페이지 등 개인화된 서비스에 접근할 수 있도록 인증 처리 구현

기술적 세부사항

- 인증이 필요없는 경로를 **whiteListUris** 에 명시하고 인증이 필요한 요청은 **LoginFilter** 를 거치도록 사용자 인증을 수행
- 세션 관리의 복잡성을 줄이기 위해 **Spring Security** 기반 **FilterChain**으로 로그인 처리 및 JWT 인증 구현
- **SecurityContextHolder의** Principal 객체로 getName 메서드를 활용하여 **사용자 인증 처리** 구현
- 비인증된 요청을 차단하기 위해 **JwtAuthenticationFilter**로 **OncePerRequestFilter** 기반 토큰 유효성 검사 구현
- 로그아웃 시 인증을 종료하기 위해 **Max-age=0**인 토큰을 발급하여 기존 토큰을 무효화

핵심 기능 2. 클린 코드를 위한 다양한 방법론들을 적용

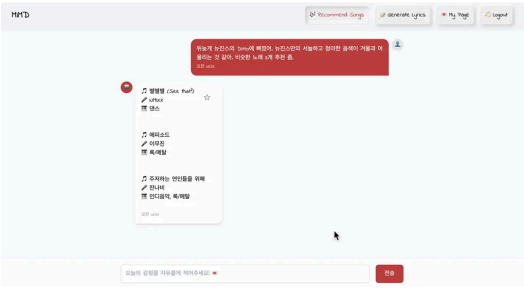
> **확장성**과 **유지보수성**을 고려한 코드 설계

기술적 세부사항

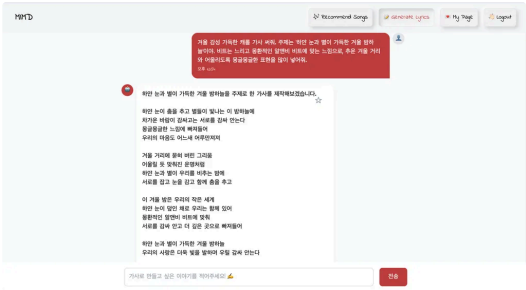
- 예외 처리를 중앙화 하기 위해 **@RestControllerAdvice** 를 활용하여 API 응답 구조 표준화
- 코드 일관성을 위해 **BaseResponse** 와 **BaseResponseCustom** 으로 응답 구조를 통일
- 복잡한 비즈니스 로직 관리을 관리하기 위해 **DDD** 를 도입하여 비즈니스 로직과 데이터 접근 로직을 분리
- 클래스 간 결합도를 낮추기 위해 상속(Inheritance) 대신 **구성(Composition)** 사용

핵심 기능 3. RestClient 통한 FastAPI 호출 및 Read Timeout 트러블슈팅

> 노래 추천 기능 선택 시 **AI가 최대 3개의 노래를 추천**하고, 가사 생성 기능 선택 시 **AI가 가사를 생성**함



프롬프팅 기반 노래 추천 기능



프롬프팅 기반 가사 생성 기능

기술적 세부사항

- FastAPI 서버와의 지속적인 통신을 위해 **Connection Pool** 을 설정하고 **RestTemplate 을 Bean 으로 등록**하여 재사용
- 요청 문자열 길이에 따른 **ReadTimeout** 문제 해결을 위해 **Spring Retry 를 사용**하여 재시도 횟수와 재시도 간격을 설정
- **ReadTimeout** 문제 방지를 위해 **calculateDynamicTimeout** 메서드를 구현하여 타임아웃을 동적으로 커스터마이징함

핵심 기능 4. 북마크 등록과 마이페이지

> AI의 채팅에 좋아요를 누르면 사용자 일기와 AI 채팅이 한쌍으로 **북마크로 등록**되고, 북마크는 마이페이지에서 조회 가능함

기술적 세부사항

- 과도한 좋아요 방지를 위해 **Bucket4j** 와 **HandlerInterceptor** 를 활용해 10초에 3회로 제한하는 **Rate Limiting** 구현
- 멱등성을 보장하기 위해 **PUT 메서드**를 사용해 북마크 등록 여부를 관리
- 한 페이지당 북마크를 3개씩 조회하기 위해 **JPA 의 @PageableDefault** 를 활용하여 **Pagination API** 구현
- 북마크 조회 시 불필요한 데이터 로드를 방지하기 위해 몇 엔티티에 **FetchMode** 설정으로 **Lazy Loading** 적용

기능 5. Frontend 구현

> React 와 TailwindCSS 를 활용하여 사용자 친화적인 UI/UX 를 구현함

기술적 세부사항

- **React**를 활용하여 주요 기능에 필요한 **공통 컴포넌트** 구현
- 회원 페이지 및 회원가입 페이지 퍼블리싱 및 API 연결
- 쿠키를 활용하여 **회원 인증 처리** API 구현
- 로그아웃 시 **클라이언트 저장소 초기화** 및 로그인 페이지로 리디렉션
- 사용자의 반복적인 요청으로 인한 RateLimit 발생 시, 서버 에러 메시지와 **10초 카운트다운 alert** 로 UX 개선

Certification

정보처리기사

2024-09-10 | 한국산업인력공단

> **CS 이론과 실무 용어에 대한 이해를 심화**하고, 실무에서 활용하기 위해 자격증 취득을 결심

- 이를 기반으로 외주 작업 당시, **애자일 스프린트 단위**로 협업하고 화면 설계서를 분석하여 **API 명세**를 주도적으로 작성함.

SQL 개발자

2024-06-21 | 한국데이터진흥원

> **ERD 설계와 최적화된 쿼리 작성 역량 강화**를 위해 자격증 취득 목표

- 소프트스퀘어드 외주 작업 당시, 다수의 테이블을 JOIN 하여 복잡한 쿼리를 작성하여 필요한 데이터를 추출할 수 있었음

Education

SK플래닛 T아카데미 ASAC 웹 풀스택 개발자 과정

2024.03 ~ 2024.09 **(6개월), 720시간**

> 웹에 대한 기초를 확립하고, 관련된 프로젝트 경험을 쌓고자 지원
> **Airbnb** 클론코딩을 진행하며 Spring Boot 기반 프로젝트 구현, 디버깅을 통해 협업 역량과 문제 해결 능력을 강화

- **웹 개론 및 CI/CD 파이프라인 - Docker, AWS, Github Actions**
  - Git 명령어 및 Working Directory 및 Staging 상태에 대한 이해와 실제 사용
  - Docker의 개념 및 사용 이유, CI/CD 절차와 Github Actions를 활용한 파이프라인 구축
- **백엔드 과정 - Spring Boot, MySQL**
  - 디자인 패턴 및 객체지향 설계원칙(SOLID) 학습
  - Java 객체 지향 프로그래밍 문법 및 JVM 의 구성 및 특징 학습
  - Spring Core 의 핵심인 IoC 와 Spring Container 특징 학습
  - Sping Boot 의 특장점 Spring Data JPA 의 연관관계 맵핑 학습
  - 어노테이션을 통한 유효성 검사와 예외처리 중앙화를 통해 코드 최적화 방법 학습

OO대학교 정보통신공학과

2020.03 ~ 2024.02 **(졸업, 휴학 1년)**

> **매학기 정시 우수자 장학금 혜택**을 받으며, 학업 과정에서 데이터베이스, 네트워크, IoT 등 기초부터 심화된 CS 지식을 쌓음

- 소켓 프로그래밍을 통해 TCP/IP 기반 통신 프로세스를 구현하고, 클라이언트-서버 모델의 기본기를 익힘
- 운영체제의 프로세스 생성, 실행, 종료까지의 생명주기를 이해하고 프로세스와 스레드의 차이점을 명확히 습득
- 시스템 호출(System Call) 및 커널과의 상호작용을 통해 리눅스의 내부 동작 원리를 학습